

IAM and Security

Users · Roles · Policies · Least Privilege · Secrets Manager · CloudTrail

AWS Tutorial · ShopFlow

02-00

Chapter Overview — IAM and Security

IAM (Identity and Access Management) is the security control plane for your entire AWS account. It answers: "Who is allowed to do what, on which resources, under what conditions?" Every API call in AWS is checked against IAM policies. For ShopFlow, IAM ensures that: the order-svc container can only read from the orders table (not delete it), developers cannot accidentally delete production databases, and no IAM key is ever stored in application code.

■ IAM in Plain English

Think of IAM as the security guard at the entrance to every AWS service. Every action (create an S3 bucket, read from DynamoDB, start an ECS task) must be approved by IAM. You define who gets access through Policies (rules), which are attached to Users, Groups, or Roles.

Example: order-svc ECS task has an IAM Role with a Policy that says: "Allow dynamodb:GetItem and dynamodb:PutItem on shopflow-cart table only." It cannot delete the table, access other tables, or call S3.

IAM Concept	What It Is	ShopFlow Example
User	A person or application with AWS credentials (access key + secret)	shopflow-developer: your IAM user for console access
Group	A collection of users that share the same permissions	Developers group: can view logs and metrics; cannot modify prod DB
Role	Temporary credentials that AWS services assume automatically (no access keys)	ECS task (order-svc) assumes this role; gets temporary AWS creds
Policy	A JSON document listing what actions are allowed or denied on which resources	AllowDynamoDBAccess: allows dynamodb:GetItem on shopflow-cart
Permission boundary	Maximum permissions a user/role can have (even if policies grant more)	Developer cannot grant themselves production DB access even if policy allows

02-01

IAM Users vs IAM Roles — The Critical Difference

This is the single most important IAM concept: use Roles for AWS services (ECS, Lambda, EC2), not Users. Users have static long-term credentials (access key + secret) that never expire and are a security risk if leaked. Roles provide temporary credentials (15 minutes to 12 hours) that are automatically rotated. ShopFlow has ZERO IAM users for application code — only IAM Roles.

	IAM User	IAM Role
Credentials	Static: Access Key + Secret Key (never expire unless manually rotated)	Temporarily issued and expire in 1-12 hours
Used by	Human developers who need console or CLI access	AWS services: ECS tasks, Lambda, EC2, CodeBuild
Security risk	High: if leaked to GitHub, attacker has permanent access	Low: temporary credentials expire; cannot be permanently stolen
ShopFlow usage	shopflow-developer user for engineer console access	order-svc ECS role, product-svc ECS role, lambda-exec role — all for application code
How to get credentials	AWS Console → IAM → Users → Security credentials	As an EC2 instance or Lambda function, temp credentials via IMDSv2 (instance metadata service)

■■■ Never create IAM users for application code

If your order-svc application uses an IAM User access key, that key is either: (1) hardcoded in the code (terrible — anyone who reads the code has full access), (2) in an environment variable (still risky — leaked in logs, error messages, CI/CD pipelines). Use IAM Roles instead — AWS automatically injects temporary credentials. No key management, no rotation, no leaks.

The Principle of Least Privilege: grant only the minimum permissions needed to do the job. ShopFlow order-svc needs to write to the orders table and publish to an SQS queue. It does NOT need to read S3, access RDS directly, or call any other service. A tight IAM policy prevents damage if the service is compromised.

■ Create a Custom IAM Policy for order-svc

1. Go to IAM in the console → Policies (left sidebar) → Create policy
2. Select JSON tab and paste the policy document (see CDK version below)
3. Click Next: Tags (optional) → Next: Review
4. Name: ShopFlowOrderServicePolicy
5. Description: Allows order-svc to write orders to DynamoDB and publish to SQS
6. Click Create policy
7. Now attach this policy to the order-svc IAM Role (next section)

IAM Policy JSON — Least Privilege for order-svc

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DynamoDBOrdersAccess",
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem", // create new order
        "dynamodb:GetItem", // read order by ID
        "dynamodb:UpdateItem", // update order status
        "dynamodb:Query" // list orders by user
      ],
      "Resource": "arn:aws:dynamodb:us-east-1:*:table/shopflow-orders"
    },
    {
      "Sid": "SQSOrderNotifications",
      "Effect": "Allow",
      "Action": ["sqs:SendMessage"],
      "Resource": "arn:aws:sqs:us-east-1:*:shopflow-order-notifications"
    }
  ]
  // Explicitly NOT granting: dynamodb>DeleteItem, s3:*, rds:*, iam:*
}
```

An IAM Role is an identity that AWS services can "assume" to get temporary credentials. When ECS starts your order-svc container, it assumes the order-svc-role and gets temporary AWS credentials injected automatically. The container code calls AWS APIs using these credentials without ever seeing an access key.

■ Create an IAM Role for ECS order-svc

1. Go to IAM → Roles → Create role
2. Trusted entity type: AWS service
3. Use case: Elastic Container Service → Elastic Container Service Task (important: Task, not EC2)
4. Click Next: Permissions
5. Search for and attach "ShopFlowOrderServicePolicy" (the policy you created in 02-02)
6. Click Next: Tags → Next: Review
7. Role name: shopflow-order-svc-role
8. Description: Role assumed by order-svc ECS tasks in production
9. Click Create role
10. Copy the Role ARN — you will reference it in CDK (arn:aws:iam::123456789012:role/shopflow-order-svc-role)

■ aws > IAM > Roles > Create role

Create role — Step 1: Select trusted entity

■ Trusted entity type

■ AWS account ■ Web identity ■ AWS service ■ Custom trust policy

■ Use case — choose the service that will use this role

■ Scroll down to "Use cases for other AWS services" and select Elastic Container Service

Elastic Container Service use case:

■ Elastic Container Service Task ■ Elastic Container Service

■ Choose Task (not EC2). Task roles are assumed by individual ECS tasks; EC2 roles are for the underlying EC2 instances.

Next: Add permissions

02-04

IAM in CDK — The Right Way

In CDK, you almost never write IAM policies manually. Instead, CDK constructs have built-in grant methods: `orderStream.grantWrite(orderServiceRole)` generates the exact DynamoDB or Kinesis permissions needed. This is safer than hand-writing ARNs and action lists, which are error-prone.

IAM Role + Permissions in CDK

```
import * as iam from "aws-cdk-lib/aws-iam";
// Create task role for order-svc ECS tasks
const orderSvcRole = new iam.Role(this, "OrderSvcRole", {
  roleName: "shopflow-order-svc-role",
  // Trust relationship: only ECS tasks can assume this role
  assumedBy: new iam.ServicePrincipal("ecs-tasks.amazonaws.com"),
  description: "Role assumed by order-svc ECS tasks",
});
// Grant order-svc permission to read/write the orders DynamoDB table
ordersTable.grantReadWriteData(orderSvcRole);
// CDK generates: dynamodb:GetItem, PutItem, UpdateItem, DeleteItem, Query, Scan
// Scoped to: arn:aws:dynamodb:us-east-1:123456789:table/shopflow-orders
```

```
// Grant order-svc permission to publish to the notification SQS queue
orderNotificationQueue.grantSendMessages(orderSvcRole);
// CDK generates: sqs:SendMessage scoped to the specific queue ARN
// Grant order-svc permission to read the database password from Secrets Manager
dbSecret.grantRead(orderSvcRole);
// CDK generates: secretsmanager:GetSecretValue scoped to the specific secret ARN
// What order-svc CANNOT do (not granted above):
// - s3:* (no S3 access)
// - iam:* (cannot modify IAM)
// - rds:* (cannot delete databases)
```

02-05

AWS Secrets Manager — Never Hardcode Passwords

ShopFlow has several secrets: Aurora database password, Stripe API key, SendGrid API key, Redis auth token. Never put these in code or environment variables. Use AWS Secrets Manager: it stores secrets encrypted, rotates them automatically, and your application fetches them at startup via an API call. If a secret leaks, rotate it in one place; all services pick up the new value automatically.

■ Store Your Aurora DB Password in Secrets Manager

1. Go to Secrets Manager in the console → Store a new secret
2. Secret type: Credentials for Amazon RDS database
3. Username: shopflow_admin
4. Password: (use auto-generate or enter a strong password)
5. Select the RDS database: choose your Aurora cluster
6. Click Next
7. Secret name: shopflow/aurora/master-credentials
8. Description: Master credentials for ShopFlow Aurora PostgreSQL cluster
9. Rotation: Enable automatic rotation → 30 days → Use a Lambda rotation function
10. Click Store — your password is now encrypted and stored

Choose secret type

Secret type

☒ Credentials for Amazon RDS database ☐ Credentials for other database ☐ API key ☐ Other type of secret

■ Credentials

User name

☒ The database master username

Password

☒ AWS generates a secure random password if you leave this empty

■ Select the database this secret will rotate for

Database

☒ shopflow-aurora-cluster (Aurora, PostgreSQL 15.4) ☐ Other database

■ When you enable automatic rotation, Secrets Manager calls the RDS API to update the password and stores the new value here automatically

[Next](#)

02-06

Reading Secrets in Spring Boot — Zero Hardcoded Values

ShopFlow Java services use the AWS SDK to fetch secrets at application startup. The database password, API keys, and auth tokens are never in code, config files, or environment variables. Spring Boot reads from Secrets Manager via a custom configuration, so injecting a new secret is transparent to the application code.

Fetching Secrets in Spring Boot (Java 21)

```
// AWS SDK for Secrets Manager
@Configuration
public class SecretsConfig {
    @Bean
    @Primary
    public DataSource dataSource() {
        // 1. Fetch secret from AWS Secrets Manager
        SecretsManagerClient client = SecretsManagerClient.builder()
            .region(Region.US_EAST_1)
            .build();
        GetSecretValueResponse response = client.getSecretValue(
            GetSecretValueRequest.builder()
                .secretId("shopflow/aurora/master-credentials")
                .build()
        );
        // 2. Parse the JSON secret value
        ObjectMapper mapper = new ObjectMapper();
        JsonNode secret = mapper.readTree(response.secretString());
        String username = secret.get("username").asText();
        String password = secret.get("password").asText();
        String host = secret.get("host").asText();
        // 3. Build the DataSource (no password in code!)
        return DataSourceBuilder.create()
            .url("jdbc:postgresql://" + host + ":5432/shopflow")
```

```

.username(username)
.password(password)
.build();
}
}

```

■ Use Spring Cloud AWS Secrets Manager integration to avoid writing this boilerplate. Add `spring-cloud-aws-starter-secrets-manager` to `pom.xml` and reference secrets as: `spring.datasource.password=${shopflow/aurora/master-credentials:password}` in `application.yml`. Spring Cloud AWS fetches the secret value automatically at startup.

02-07

IAM Permission Boundaries — Guard Rails for Developers

Permission Boundaries limit the maximum permissions a user or role can have, even if attached policies grant more. ShopFlow uses permission boundaries so developers can create IAM roles for their services but cannot grant themselves or their services access to production databases or payment systems.

Without Permission Boundary	With Permission Boundary
Developer creates IAM role for their service with AdministratorAccess	Developer creates role but AWS limits max permissions to a safe set (no AdministratorAccess)
Compromised developer account → attacker escalates to full compromised AWS	Compromised developer account → attacker still bounded by permission boundary
Accidental policy grants too much access	Even over-permissive policy is bounded; actual access = intersection of policy and boundary

Permission Boundary CDK

```

// Create the boundary: max allowed permissions for any developer-created role
const devBoundary = new iam.ManagedPolicy(this, "DevBoundary", {
  managedPolicyName: "shopflow-developer-boundary",
  statements: [
    new iam.PolicyStatement({
      effect: iam.Effect.ALLOW,
      actions: ["s3:*", "dynamodb:*", "lambda:*", "logs:*", "cloudwatch:*",
        "sqs:*", "sns:*", "kinesis:*", "secretsmanager:GetSecretValue"],
      resources: ["*"],
    }),
    new iam.PolicyStatement({
      effect: iam.Effect.DENY,
      actions: ["iam:*", "rds:DeleteDBCluster", "rds:DropDBTable",
        "ec2:DeleteVpc", "cloudformation:DeleteStack"],
      resources: ["*"],
    }),
  ],
});
// Attach to developer roles: permissionsBoundary: devBoundary

```

02-08

AWS Organizations and Service Control Policies

As ShopFlow grows, you will have multiple AWS accounts: Production, Staging, Development, Security. AWS Organizations groups accounts into an organization with a management account. Service Control Policies (SCPs) apply to entire accounts — they override even `AdministratorAccess`. ShopFlow uses SCPs to prevent anyone in the Dev account from launching expensive instance types.

Create service control policy

■ Policy details

Policy name

ShopFlowDevAccountRestrictions

Description

Prevent expensive resources in Dev account

■ Policy content — paste JSON

■ The JSON SCP below restricts the Dev account from launching large instance types and Aurora production clusters

■ SCP applies to ALL users in the account, including root. Even AdministratorAccess cannot override an SCP deny.

Create policy

SCP — Restrict Dev Account

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyExpensiveInstances",
    "Effect": "Deny",
    "Action": ["ec2:RunInstances", "rds:CreateDBCluster"],
    "Resource": "*",
    "Condition": {"StringLike": {
      "ec2:InstanceType": ["*.4xlarge", "*.8xlarge", "*.16xlarge", "*.metal"],
      "rds:DatabaseClass": ["db.r6g.4xlarge", "db.r6g.8xlarge"]
    }}
  ]
}
```

02-09

Enabling AWS CloudTrail — Audit Log for Everything

CloudTrail records every API call made to your AWS account: who called what, when, from which IP, with what parameters. Essential for security auditing. ShopFlow enables CloudTrail on day one — required for compliance (PCI DSS, SOC 2). If someone accidentally deletes a production resource, CloudTrail tells you exactly who did it and when.

■ Enable CloudTrail for ShopFlow

1. Search "CloudTrail" in the console
2. Click Create trail
3. Trail name: shopflow-audit-trail
4. Storage location: Create new S3 bucket → shopflow-cloudtrail-logs-ACCOUNTID
5. Log file SSE-KMS encryption: enabled (create a new KMS key)
6. Enable CloudWatch Logs: create a new log group /aws/cloudtrail/shopflow
7. Events to log: Management events ■ (all API calls) + Data events ■ (S3 object-level)
8. Click Create trail — CloudTrail is now recording all API calls

CloudTrail Event Type	What Gets Recorded	ShopFlow Use Case
Management events	CreateBucket, LaunchInstance, DeleteTable, CreateTable	Detect: "Was someone deleting a production DynamoDB table?" or "Was someone launching a new EC2 instance?"
Data events (S3)	GetObject, PutObject, DeleteObject on specific S3 buckets	Detect: "Did anyone download all customer data from the prod bucket?"
Data events (Lambda)	Invoke calls on Lambda functions	Track: which service called which Lambda function and how often
Insights events	Unusual API call patterns detected by ML	Alert: unusual spike in DescribeInstances calls at 3am (credentials compromised?)

02-10

Security Best Practices Checklist for ShopFlow

Before launching any ShopFlow component to production, run through this security checklist. These are the most common security mistakes at AWS-hosted companies, learned from real incidents.

Security Control	Priority	Console Check	CDK Check
MFA enabled on all IAM users	Critical	IAM → Users → each user → MFA assigned	N/A (human action)
Root account has no access keys	Critical	IAM → Security status → No root access keys	N/A (human action)
CloudTrail enabled in all regions	Critical	CloudTrail → Trails → Multi-region: Yes	<code>new cloudtrail.Trail(this,"T",{isMultiRegion: true})</code>
No hardcoded AWS credentials in code	Critical	Search GitHub repos for AKIA (access key prefix)	Use IAM Roles; never <code>iam.User</code> for service accounts
All S3 buckets block public access	High	S3 → Block Public Access settings → All enabled	<code>blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL</code>
VPC security groups: no 0.0.0.0/0 ingress on port 22	High	EC2 → Security Groups → review inbound rules	Use SSM Session Manager instead of SSH
Secrets in Secrets Manager (not env vars)	High	Secrets Manager → at least DB + API key secrets	<code>new secretsmanager.Secret(this,"DBSecret")</code>
RDS not publicly accessible	High	RDS → Databases → Connectivity → Publicly accessible: No	<code>publiclyAccessible: false</code>
Encryption at rest on all data stores	Medium	RDS/DynamoDB/S3/ElastiCache → encryption settings	<code>storageEncrypted: true</code> on all data constructs
AWS Config enabled (compliance monitoring)	Medium	AWS Config → Settings → Recording enabled	Automated via SecurityHub or manually

02-11

Setting Up AWS IAM Identity Center (SSO)

AWS IAM Identity Center (formerly AWS SSO) is the recommended way to manage access when you have multiple AWS accounts or multiple team members. Instead of creating separate IAM users in each account, IAM Identity Center provides a single sign-on portal where developers log in with company credentials (Google Workspace, Okta, Active Directory) and get time-limited console access.

■ Enable IAM Identity Center for ShopFlow

1. Search "IAM Identity Center" in the console → Enable IAM Identity Center
2. Choose identity source: Identity Center directory (built-in) or external IdP
3. Create a Permission Set: ShopFlowDeveloperAccess → attach PowerUserAccess managed policy
4. Create a Permission Set: ShopFlowReadOnlyAccess → attach ReadOnlyAccess managed policy
5. Add users: enter developer email addresses
6. Assign users to accounts: Dev account → ShopFlowDeveloperAccess; Prod account → ShopFlowReadOnlyAccess
7. Send invitation email to each developer
8. Developers visit the SSO portal URL (shown in IAM Identity Center dashboard)
9. They log in once and see all accounts they have access to — no static IAM user keys needed

■ Why IAM Identity Center?

With plain IAM Users, each developer has a separate username/password in each AWS account. With Identity Center: one login gives access to all accounts you are authorized for. The portal issues short-lived credentials (8-hour sessions). If someone leaves the company, you remove them from Identity Center once — access to all AWS accounts is revoked immediately.

Example: A developer logs into the SSO portal at <https://shopflow.awsapps.com/start>, sees "Dev account" and "Staging account" tiles, clicks "Dev account → Developer access", and gets console access or CLI credentials valid for 8 hours.

Feature	IAM Users	IAM Identity Center
Login	Separate password per account (one account = one login credential)	Single login for all accounts via SSO portal or CLI (aws sso login)
Credential lifetime	Static keys; never expire unless manually rotated	Time-limited: console sessions expire in 1-12 hours (configurable)
Revocation	Must delete IAM user in every account separately	Simply remove from Identity Center once; all account access revoked immediately
MFA	Configure per user per account	Centrally enforced; one MFA device covers all accounts
Developer experience	aws configure → enter key ID and secret	aws sso login → browser opens; select account; credentials auto-generated

02-12

IAM Roles for Cross-Account Access

ShopFlow uses separate AWS accounts for Dev, Staging, and Production. A CodePipeline in the Tools account deploys to the Production account. This requires cross-account IAM trust: the CodePipeline role in Tools is trusted by a deployment role in Production. Understanding cross-account roles is essential for multi-account AWS architectures.

Account	Purpose	What It Contains
Management / Root	AWS Organizations root; billing aggregation	No workloads; just SCPs and billing. Nobody works here directly.
Security	Centralized security tooling	CloudTrail logs, GuardDuty, Security Hub, CloudWatch centralized logs
Tools	CI/CD pipeline infrastructure	CodePipeline, CodeBuild, ECR repositories, CDK deployments
Dev (111111111111)	Developer sandbox	ShopFlow full stack, cheap instance sizes, auto-shutdown at night
Staging (222222222222)	Pre-production testing	Production-like, but smaller. QA team and integration tests run here.
Production (333333333333)	Customer traffic	Highest IAM security; changes only via approved CodePipeline deployments

Cross-Account Deployment Role CDK

```
// In the PRODUCTION account CDK stack:
const deploymentRole = new iam.Role(this, "DeploymentRole", {
  roleName: "shopflow-pipeline-deployment-role",
  // Trust the Tools account CodePipeline role to assume this role
  assumedBy: new iam.AccountPrincipal("111111111111"), // Tools account ID
});
// Grant deployment role permission to update ECS services and CloudFormation
deploymentRole.addManagedPolicy(
  iam.ManagedPolicy.fromAwsManagedPolicyName("AWSCloudFormationFullAccess")
);
ordersTable.grantReadWriteData(deploymentRole);
```

02-13

GuardDuty — Threat Detection

AWS GuardDuty is a managed threat detection service that continuously monitors your AWS account for suspicious activity. It analyzes CloudTrail logs, VPC Flow Logs, and DNS logs using ML to detect: compromised EC2 instances, unauthorized API calls, cryptocurrency mining, and credential exfiltration. ShopFlow enables GuardDuty across all accounts from day one — it costs pennies per day and has caught real intrusions.

■ Enable GuardDuty for ShopFlow

1. Search "GuardDuty" in the console → Get Started → Enable GuardDuty
2. GuardDuty is now analyzing your account automatically — no agents to install
3. Click Settings → Finding export options → configure S3 export (for long-term retention)
4. Click Findings → you will see any current threats (new accounts often have zero)
5. Click Protection plans → enable EKS Audit Log Monitoring, RDS Protection, S3 Protection
6. Set up EventBridge rule: when GuardDuty finding severity >= HIGH → SNS → email alert
7. In Organizations: enable GuardDuty for all member accounts from the management account

GuardDuty Finding	What It Means for ShopFlow	Response
UnauthorizedAccess:EC2/SSHBruteForce	Someone is trying to SSH into your EC2/EC3 instances	Block by guessing passwords; use SSM Session Manager
Recon:IAMUser/MaliciousIPCaller	An IAM user made API calls from a known malicious IP	Immediately revoke user credentials; check CloudTrail for actions
CryptoCurrency:EC2/BitcoinTool	A container is mining cryptocurrency — indicates compromise	Immediately stop EC2 task; investigate container image
Exfiltration:S3/MaliciousIPCaller	Data was downloaded from S3 to a suspicious IP	Check what data was accessed; enable S3 access logging
Policy:IAMUser/RootCredentialUsage	Root account was used (should almost never happen)	Investigate immediately; rotate root credentials; check w

02-14

AWS Security Hub — Centralized Security Score

Security Hub aggregates findings from GuardDuty, Inspector (vulnerability scanning), Macie (sensitive data in S3), IAM Access Analyzer, and third-party tools. It provides a security score and checks your account against the AWS Foundational Security Best Practices standard. ShopFlow aims for 90%+ security score before going to production.

■ aws > Security Hub > Summary

Security Hub — ShopFlow Account Overview

■ Security score

■ AWS Foundational Security Best Practices: Security score: 73% ↑ 4% this week

■ Failed controls (top issues to fix)

- [CRITICAL] IAM.1 — IAM policies should not allow full "*" admin privileges (2 resources)
- [HIGH] S3.2 — S3 buckets should prohibit public read access (1 resource: shopflow-legacy-uploads)
- [HIGH] RDS.3 — RDS DB instances should have deletion protection enabled (1 resource)
- [MEDIUM] CloudTrail.2 — CloudTrail should have encryption at rest enabled (1 resource)

■ Passed controls

■ 47 of 64 controls passing — click [View all controls](#) to see details and remediation steps

[View all controls](#)

■ Security Hub provides one-click remediation for many findings. Click on a failed control, read the remediation steps, and for many controls there is a "Remediate" button that applies the fix automatically. For ShopFlow, remediate all Critical and High findings before going live.

IAM Access Analyzer scans your IAM policies, S3 bucket policies, KMS key policies, and Lambda function policies to find resources that are accessible from outside your AWS account. For ShopFlow, this detects: S3 buckets accidentally made public, cross-account access not in your design, and IAM roles that can be assumed by third parties.

■ Enable IAM Access Analyzer

1. Go to IAM → Access Analyzer (bottom of left sidebar) → Create analyzer
2. Analyzer name: shopflow-account-analyzer
3. Zone of trust: Current account (findings = anything accessible outside this account)
4. Click Create analyzer — scan runs automatically
5. Review findings: each finding shows the resource, who has external access, and what actions are allowed
6. For each finding: click Active → Archive (if intended) or Resolve (if unintended)
7. Resolve unintended S3 public access: update the bucket policy to restrict to VPC only

Access Analyzer Finding Type	Example	ShopFlow Action
S3 Bucket Policy — external access	shopflow-customer-uploads allows s3:GetObject publicly	Remove publicly; enable S3 Block Public Access; use
IAM Role — cross-account trust	shopflow-order-svc-role has trust policy allowing any account	Restrict trust to specific account IDs or AWS Organizations
KMS Key — external usage	product-encryption-key allows Decrypt from VeriDict	Verify if VeriDict is a known partner; if not, remove the e
Lambda Function URL — public	order-processing Lambda has a public function URL	And use AWS IAM. Require sigv4 signing for all Lambda U

IAM Conditions add context to policy statements: "allow this action, but only if the request comes from the VPC, uses MFA, is in us-east-1, and happens during business hours." Conditions transform flat allow/deny decisions into fine-grained, context-aware access control. ShopFlow uses conditions on all sensitive operations.

Common IAM Condition Operators for ShopFlow

```
// Condition 1: Require MFA for sensitive admin operations
{
  "Effect": "Allow",
  "Action": ["iam:*", "cloudformation:Delete*"],
  "Resource": "*",
  "Condition": {"Bool": {"aws:MultiFactorAuthPresent": "true"}}
}

// Condition 2: Restrict S3 access to requests from the ShopFlow VPC only
{
  "Effect": "Deny",
  "Action": "s3:*",
  "Resource": "arn:aws:s3:::shopflow-customer-data/*",
  "Condition": {"StringNotEquals": {"aws:SourceVpce": "vpce-0abc123def456"}}
}

// The above denies S3 access from anywhere except the VPC endpoint
// Condition 3: Restrict which regions services can run in
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {"StringNotEquals": {"aws:RequestedRegion": ["us-east-1", "us-west-2"]}}
```

}

Condition Key	Description	ShopFlow Use Case
aws:MultiFactorAuthPresent	Is the caller using MFA?	Require MFA for IAM changes, CloudFormation deletes, RDS
aws:SourceVpce	Which VPC endpoint is the request coming from?	Restrict S3 product data access to requests from the ShopFlow
aws:RequestedRegion	Which AWS region is the call targeting?	Deny all service launches outside us-east-1 and us-west-2
aws:RequestTag	Tags on the resource being requested	Allow developers to stop only ECS tasks tagged with their own
aws:CurrentTime	Time of day the request is made	Deny production RDS deletion outside business hours (00:00-
aws:PrincipalOrgID	Is the caller in the same AWS Organization?	Allow cross-account S3 access only to principals within the SH

02-17

Hands-On Lab — Secure ShopFlow from Scratch

This lab walks you through securing a new ShopFlow AWS account from zero. By the end you will have: root account locked down, CloudTrail enabled, GuardDuty running, an IAM user for yourself, and a test IAM role for an ECS service. Estimated time: 45 minutes.

Step	Action	Verification
1	Log in to AWS console as root → My Account → Activate MFA (Windows MFA) → Security and authentication	My Account → Security and authentication: Root MFA enabled ■
2	IAM → Account Settings → Password policy: min 14 chars, require uppercase letters → System password policy	IAM → Account Settings → System password policy active ■
3	CloudTrail → Create trail → shopflow-audit-trail → multi-region CloudTrail S3 Trail → CloudWatch logs	CloudTrail → shopflow-audit-trail → Status: Logging ■
4	GuardDuty → Enable → note the S3 protection, RDS protection	GuardDuty → Enable → Monitoring status: Enabled ■
5	IAM → Users → Create user → your name → Console access only → Developers group → IAM user access	IAM → Users → your name → Developers group IAM user access ■
6	IAM → Roles → Create role → ECS Task → name: test-order-svc-role → ARN displayed copy it	IAM → Roles → test-order-svc-role → Role ARN displayed copy it ■
7	IAM → Policies → Create policy → JSON → paste the order-service-policy-0202-SecretP	IAM → Policies → test-order-svc-role → Policy created ■
8	Attach policy to role: IAM → Roles → test-order-svc-role → Attach policies → Attach ShopFlow	IAM → Roles → test-order-svc-role → Attach ShopFlow ServicePolicy
9	Secrets Manager → Store secret → Other type → Key: db_password → Value: test123456789 → Name: shopflow/test/db	Secrets Manager → test123456789 → Name: shopflow/test/db
10	IAM Access Analyzer → Create analyzer → current account → Analyze any findings	IAM Access Analyzer → test123456789 → Analyze any findings shown (archive any intentional ones) ■

02-18

ShopFlow IAM Architecture — All Services and Roles

Here is the complete IAM role architecture for all 8 ShopFlow microservices. Each service has its own role with the minimum permissions needed. No role has permissions it does not use. Each role is scoped to specific resource ARNs, not wildcards.

ECS Service / Lambda	IAM Role Name	Key Permissions
order-svc	shopflow-order-svc-role	dynamodb:PutItem/GetItem/UpdateItem/Query on orders table; sqs:SendMessage
product-svc	shopflow-product-svc-role	dynamodb:GetItem/Query on products table; s3:GetObject on product-images
user-svc	shopflow-user-svc-role	cognito-idp:AdminGetUser/ListUsers; secretsmanager:GetSecretValue for
inventory-svc	shopflow-inventory-svc-role	dynamodb:UpdateItem/GetItem on inventory table; sqs:ReceiveMessage/Dele
payment-svc	shopflow-payment-svc-role	secretsmanager:GetSecretValue for Stripe key; kms:Decrypt for payment
notification-svc	shopflow-notification-svc-role	sqs:ReceiveMessage/DeleteMessage from notification-queue; ses:SendE
search-svc	shopflow-search-svc-role	es:ESHttpGet/ESHttpPost on shopflow-search OpenSearch domain; logs:Log
analytics-lambda	shopflow-analytics-lambda-role	kinesis:GetRecords/GetShardIterator on event-stream; s3:PutObject on an

ShopFlow IAM Hierarchy — All Accounts and Roles

AWS Organization (shopflow.com)

Management Account — billing and SCPs only

Security Account — GuardDuty, CloudTrail, Security Hub

Tools Account — CodePipeline, ECR, CDK

Production Account (333333333333)

IAM Roles (Services): order-svc, product-svc, user-svc, inventory-svc

IAM Roles (Services): payment-svc, notification-svc, search-svc, analytics-

IAM Roles (Infra): ecs-task-execution-role — pull ECR images, write CloudWa

IAM Roles (Infra): pipeline-deployment-role — cross-account CodePipeline de

IAM Roles (Infra): cdk-deploy-role — apply CloudFormation changes via CDK

02-19

Rotating IAM Credentials and Key Best Practices

IAM access keys for human users (not service roles) must be rotated regularly. Leaked access keys are the #1 cause of AWS account compromise. ShopFlow policy: all developer IAM access keys rotate every 90 days, enforced via IAM Access Analyzer and AWS Config. Keys are deactivated before deletion to allow 24 hours for systems to catch up.

■ Rotate Your IAM Access Key (every 90 days)

1. IAM → Users → (your username) → Security credentials tab
2. Under "Access keys" — note the age of your current key (Created column)
3. Click Create access key → choose CLI as the use case → download the new .csv file
4. Update your local ~/.aws/credentials with the new key: aws configure
5. Verify the new key works: aws sts get-caller-identity (should return your IAM ARN)
6. Return to IAM → Users → your user → Security credentials → deactivate the OLD key (Status: Active → Inactive)
7. Wait 24 hours to ensure no systems are using the old key (check CloudTrail for old key usage)
8. After 24 hours: delete the old key — click the X next to the deactivated key
9. Add a reminder in your calendar for 90 days from today to repeat this process

IAM Key Hygiene Rule	Why It Matters	How to Enforce
Never commit access keys to Git	git history is permanent; rotated keys found in git history are a security risk	Use git-secrets or GitHub secret scanning; pre-commit hooks
Rotate every 90 days maximum	The longer a key lives, the higher the chance it is leaked or compromised	AWS Config rule: access-keys-rotated with maxAccessKeyAge
Use separate keys for separate systems	If one system is compromised, only its key is exposed	One key per developer per environment; never share keys
Prefer IAM Identity Center over static keys	Static keys expire in 1-12 hours; cannot be rotated by users	Migrate all developers from static keys to aws sso login with CLI
Alert on unused keys	Keys with no activity in 45+ days are a security risk	AWS Config + Lambda: disable keys with no usage in 45 days

Bringing it all together: this is the complete security architecture that protects ShopFlow and 1 million customers. Every layer has a specific role; no single layer is trusted to work alone (defense in depth).

Security Layer	Technology	What It Protects Against
Network perimeter	VPC + Security Groups + NACLs	Unauthorized network access to databases and services from the internet
Identity and access	IAM Roles (least privilege) + IAM Identity Center (SSO)	Unauthorized API calls; credential theft; privilege escalation
Secrets management	AWS Secrets Manager (rotation enabled)	Hardcoded passwords; static credentials in code, config, or environment
Application perimeter	WAF + API Gateway + Cognito	SQL injection, XSS, brute-force attacks, unauthenticated API access
Data at rest	KMS encryption on RDS, DynamoDB, S3, Elastic File System	Data loss; unauthorized storage layer; rogue insider access to raw storage
Data in transit	TLS 1.3 on all connections, VPC endpoints for AWS APIs	Man-in-the-middle attacks; credential sniffing on network
Threat detection	GuardDuty (ML anomaly detection), Security Hub (centralized)	Cloud misconfigurations; cryptocurrency mining; data exfiltration attempts
Audit and compliance	CloudTrail (all API calls) + AWS Config (resource configuration)	Unauthorized changes; compliance violations; forensic investigation
Vulnerability scanning	Inspector (EC2/Lambda/ECR), Macie (S3 sensitive data)	Unpatched CVEs in containers; customer PII accidentally stored in wrong buckets
Permission guardrails	Permission Boundaries + SCPs (Organization-wide)	Developer error granting excessive permissions; account-level lockdown

■ ShopFlow Security Baseline

This is the minimum security configuration for any AWS workload handling customer data. Set up all 10 layers before going live. The cost of these security controls is approximately \$50-150/month for a medium-sized application — far less than the cost of a breach (average AWS account compromise costs \$300,000 in incident response, lost revenue, and regulatory fines, per AWS security team estimates).

- ✓ Use IAM Roles for all AWS services (ECS, Lambda, EC2) — roles provide temporary credentials that auto-rotate; never use IAM user access keys for application code
- ✓ Apply least-privilege: start with zero permissions, add only what is needed — it is easier to add permissions later than to discover a security breach caused by over-permissive access
- ✓ Enable CloudTrail in all regions on day one — you cannot investigate a security incident that happened before CloudTrail was enabled
- ✓ Store all secrets in AWS Secrets Manager with automatic rotation enabled — database passwords that never rotate are a major security risk
- ✓ Use permission boundaries for developer IAM roles — prevents privilege escalation even if a developer accidentally attaches AdministratorAccess to their service role
- ✓ Enable MFA on every IAM user without exception — a stolen password without MFA gives full console access to your production environment
- ✓ Use CDK grantXxx() methods (grantRead, grantWrite, grantSendMessages) instead of writing ARNs manually — CDK generates the correct minimum permissions automatically
- Never use AdministratorAccess policy on ECS task roles or Lambda execution roles — each service gets only the permissions it needs; AdministratorAccess is appropriate only for human admin users
- Do not use inline policies on ECS task roles — use managed policies that can be reviewed, versioned, and reused across services
- Never commit .aws/credentials or AWS access keys to Git — use git-secrets or AWS CodeGuru to scan for credential patterns in all commits

Question	Answer
What is the difference between IAM Users, Groups, Roles and Policies?	Users, Groups, Roles are all principal types. User = unique set of static credentials. Group = collection of users sharing policies. Role = collection of permissions that can be assumed by a principal.
How does an ECS task get AWS credentials to use to access other AWS services?	ECS tasks get temporary credentials via the Task Metadata endpoint (IMDSv2). The task role ARN is specified in the task definition.
What is the difference between resource-based and identity-based policies?	Identity-based: attached to IAM principals; defines what that identity can DO. Resource-based: attached to AWS resources; defines who can access the resource.
How does Secrets Manager rotation work?	When rotation is triggered (on schedule or manually), Secrets Manager calls a Lambda function that: (1) generates new secret, (2) updates secret with new value, (3) removes old secret.